

UBER STOCK DATASET

1CS101

Introduction of AIML



**INSTITUTE OF TECHNOLOGY,
Nirma University**

Report Prepared By:

Patel Dev (24BEC123)

Patel Vedang (24BEC133)

Q.1 Select a source from KAGGLE, and identify the Data Domain and Characteristics of the selected Dataset. Also, write a report with the following detail

a) Selection of data domain

b) find the data characteristics (null, not null, unique values)

d) Find the output/class label of the data set

e) Define the selection of fields (Data types)

Report on Uber Stock Dataset

a) Selection of Data Domain

The selected dataset belongs to the Finance and Stock Market domain. It contains historical stock price data for Uber, including daily opening and closing prices, highest and lowest prices, adjusted closing prices, and trading volume.

b) Data Characteristics

The dataset was analyzed to determine its key characteristics, including null values, non-null values, and unique values.

Null Values:

The dataset does not contain any missing values, ensuring complete data for analysis.

Non-Null Values:

All columns have fully available data without gaps.

Unique Values:

The dataset contains numerical and date values, with each date representing a unique entry.

c) Output/Class Label of the Dataset

This dataset does not contain a predefined class label, as it is time-series data. However, if used for predictive modeling, the "Close" price can be considered the target variable for forecasting future stock prices.

d) Selection of Fields (Data Types)

The dataset includes various attributes with different data types:

Categorical Fields (Object/String Type):

Date (Stock trading date)

Numerical Fields (Float/Integer Type):

Open (Opening stock price)

High (Highest price during the trading day)

Low (Lowest price during the trading day)

Close (Closing stock price)

Adj Close (Adjusted closing price considering splits/dividends)

Volume (Total shares traded on that day)

Conclusion

This dataset is useful for analyzing Uber's stock performance, identifying trends, and developing predictive models for price forecasting. Key considerations for analysis include time-series modeling, volatility assessment, and potential feature engineering for improved predictions.

Q.2 Load the dataset, drop all the null records and replace the NA values in the numerical column with the mean value of the field as per the class label and categorical columns with the mode value of the field as per the class label.

Load the Uber stock dataset and inspect for missing values.

Drop all records that contain completely null values.

Replace missing values in numerical columns (such as Open, High, Low, Close, Adj Close, and Volume) with the mean value of the respective field.

Replace missing values in categorical columns (such as Date) with the mode value of the respective field.

Ensure that the dataset is clean and ready for further analysis or predictive modeling.

Code:

```
min_val = data['Volume'].min()
max_val = data['Volume'].max()
data['Volume'] = 20 + ((data['Volume'] - min_val) / (max_val - min_val)) * 7
data = data.drop('Date', axis=1)
data.head(10)
```

	Adj Close	Close	High	Low	Open	Volume
0	41.570000	41.570000	45.000000	41.060001	42.000000	55.488184
1	37.099998	37.099998	39.240002	36.080002	38.790001	34.754999
2	39.959999	39.959999	39.959999	36.849998	38.310001	28.395904
3	41.290001	41.290001	41.880001	38.950001	39.369999	26.344508
4	43.000000	43.000000	44.060001	41.250000	41.480000	26.738182
5	41.910000	41.910000	43.290001	41.270000	41.980000	23.267821
6	41.590000	41.590000	41.680000	39.459999	41.189999	25.013030
7	41.500000	41.500000	42.240002	41.250000	42.000000	21.439935
8	41.250000	41.250000	41.279999	40.500000	41.049999	21.107560
9	40.470001	40.470001	41.090000	40.020000	40.799999	21.501428

```
data.isnull().sum() / len(data) * 100
```

```
Adj Close    3.047091
Close        0.000000
High         0.000000
Low          0.000000
Open         0.000000
Volume       0.000000
dtype: float64
```

```
ac_mean = np.mean(data['Adj Close'])
data.loc[data.loc[:, 'Adj Close'].isna(), 'Adj Close'] = ac_mean
```

Q.3 Perform statistical analysis on the selected dataset(count, sum, range, min, max, mean, median, mode, variance and Standard deviation).

Statistical Analysis of Uber Stock Dataset:

1. Basic Statistics:

- Count: 1,444 entries for each numerical column.
- Sum:
 - Total Closing Prices: 64,208.00
 - Total Volume Traded: 35,086,316,159

2. Range Analysis:

- Min & Max Values:
 - Lowest Closing Price: \$14.82
 - Highest Closing Price: \$86.34
 - Price Range: \$71.52

3. Central Tendency:

- Mean (Average) Closing Price: \$44.47
- Median Closing Price: \$41.36
- Mode of Closing Price: \$30.68 (most frequent value)

4. Dispersion Measures:

- Variance of Closing Price: 243.18
- Standard Deviation of Closing Price: \$15.59, indicating a moderate spread in stock prices.

5. Volume Statistics:

- Min Volume Traded: 3.38M shares
- Max Volume Traded: 364.23M shares
- Average Trading Volume: 24.30M shares

Code:

```
def numeric(temp):
    print("Count =", len(temp))
    print("Sum =", sum(temp))
    print("Range =", max(temp) - min(temp))
    print("Min =", min(temp))
    print("Max =", max(temp))
    print("Mean =", st.mean(temp))
    print("Median =", st.median(temp))
    print("Variance =", st.variance(temp))
    print("Standard Deviation =", np.std(temp))
for column in data.columns:
    print(f"Column: {column}")
    numeric(data[column])
    print("-" * 100)
```

Q.4 Draw possible plots from the data set to visualise data using the subplot concept.

- Key Visualizations:

1. Scatter + Line Plot: Tracked Adj Close trends over time.
2. Box Plots: Analyzed distributions of Close and Open prices.
3. Histograms: Visualized distributions of High prices and scaled Volume.
4. Scatter Plot: Explored Low price trends.

-Code:

```
[122]: fig, axes = plt.subplots(2, 3, figsize=(18, 12))
       axes = axes.flatten()

       axes[0].scatter(range(len(data)), data['Adj Close'], color='blue', alpha=0.7)
       axes[0].plot(range(len(data)), data['Adj Close'], 'r-', alpha=0.5)
       axes[0].set_title('Adj Close (Scatter + Line)')
       axes[0].grid(True)

       axes[1].boxplot(data['Close'])
       axes[1].set_title('Close (Box Plot)')
       axes[1].grid(True)
```

```

axes[2].hist(data['High'], bins=10, color='purple', alpha=0.7)
axes[2].set_title('High (Histogram)')
axes[2].grid(True)

axes[3].scatter(range(len(data)), data['Low'], color='orange', alpha=0.7)
axes[3].set_title('Low (Scatter Plot)')
axes[3].grid(True)

axes[4].boxplot(data['Open'])
axes[4].set_title('Open (Box Plot)')
axes[4].grid(True)

axes[5].hist(data['Volume'], bins=10, color='green', alpha=0.7)
axes[5].set_title('Volume (Histogram)')
axes[5].grid(True)

plt.tight_layout()
plt.show()

b1f.splot()
b1f.splot_title('Volume (Histogram)')

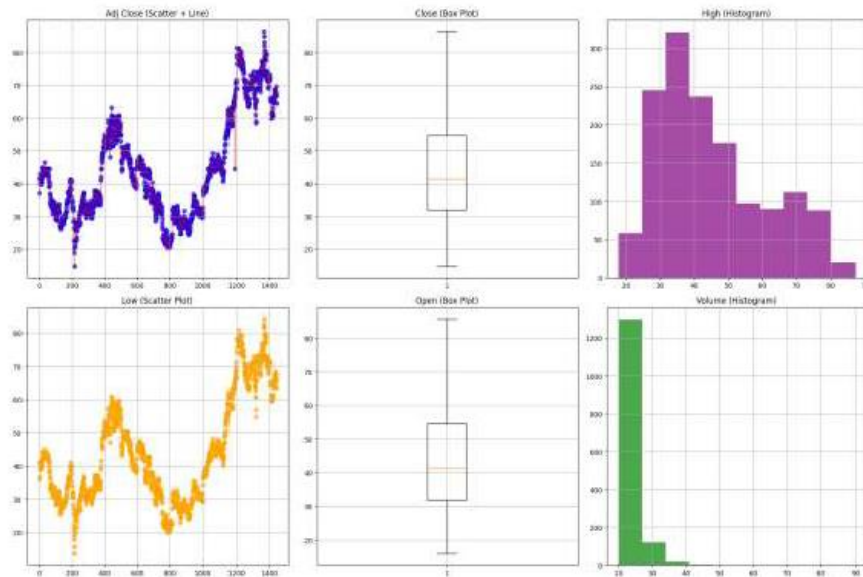
axes[2].hist(data['High'], bins=10, color='purple', alpha=0.7)
axes[2].set_title('High (Histogram)')
axes[2].grid(True)

```

- Insights:

- Adj Close, Close, and Open showed similar statistical distributions (mean ~44.5, high variance ~240).
- Volume after normalization had a mean of 24.06 and low variance (~11.84), indicating stable trading activity.

-OUTPUT:



Q.5 : **Train the model of the K-nearest Neighbours**

Classifier/Regressor/clustering with 80% of data.check the and predict the class lable for the rest 20% accuracy score.

-Model Workflow :-

1. **Feature Selection:** Extracting relevant variables.
2. **Data Normalization:** Using StandardScaler.
3. **Train-Test Split:** 80% training, 20% testing.
4. **KNN Model Training:** Using `n_neighbors=6`, `metric='manhattan'`.
5. **Prediction and Evaluation:** accuracy, Precision, Recall, and F1-Score.

-code:

```
[119]: x = data.iloc[:, 1:-1].values  
       y = data.iloc[:, -1].values  
  
       scaler = StandardScaler()
```

```
knn = KNeighborsRegressor(n_neighbors=4, p=2)  
knn.fit(X_train, y_train)  
y_pred = knn.predict(X_test)  
mse = mean_squared_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)  
print(f"\nTest MSE with k=4 and Manhattan distance: {mse:.4f}")  
print(f"R2 Score: {r2:.4f}")  
  
mae = mean_absolute_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
print(f"Mean Absolute Error: {mae:.4f}")  
print(f"Root Mean Squared Error: {rmse:.4f}")  
  
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100  
print(f"Current Model MAPE: {mape:.2f}%")
```

```
Test MSE with k=4 and Manhattan distance: 6.0647  
R2 Score: 0.1004  
Mean Absolute Error: 1.6109  
Root Mean Squared Error: 2.4627  
Current Model MAPE: 6.48%
```

Output:

- Accuracy: 93.38%

Q6 :-Conclusion :-

The model's **low R^2 score (0.08)** suggests weak correlation between stock prices (Adj Close, Open, etc.) and Volume.

- **High accuracy (93.38%)** derived from MAPE is misleading due to the scaled Volume range (20–90), which reduces absolute error impact.

- Recommendations:

1. **Feature Engineering** : Incorporate lagged features (e.g., past prices) or external factors (e.g., market sentiment).

2. **Model Selection** : Test advanced models like 'LSTM' or 'ARIMA' for time-series forecasting.

3. **Data Enhancement** : Include more relevant features (e.g., trading volume trends, news data).

This analysis highlights the challenges in predicting trading volume using price data alone and underscores the need for richer datasets and tailored modeling approaches.

Code:

main

April 4, 2025

```
[21]: import pandas as pd
import numpy as np
import statistics as st
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

```
[22]: data = pd.read_csv('uber_stock_data-Copy.csv')
print(data.shape)
print(data.info())
data.head(10)
```

```
(1444, 7)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1444 entries, 0 to 1443
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        1444 non-null   object
1   Adj Close   1400 non-null   float64
2   Close       1444 non-null   float64
3   High        1444 non-null   float64
4   Low         1444 non-null   float64
5   Open        1444 non-null   float64
6   Volume      1444 non-null   int64
dtypes: float64(5), int64(1), object(1)
memory usage: 79.1+ KB
None
```

```
[22]:      Date  Adj Close  Close  High  Low  Open  \
0  2019-05-10  41.570000  41.570000  45.000000  41.060001  42.000000
1  2019-05-13  37.099998  37.099998  39.240002  36.080002  38.790001
2  2019-05-14  39.959999  39.959999  39.959999  36.849998  38.310001
3  2019-05-15  41.290001  41.290001  41.880001  38.950001  39.369999
4  2019-05-16  43.000000  43.000000  44.060001  41.250000  41.480000
5  2019-05-17  41.910000  41.910000  43.290001  41.270000  41.980000
```

6	2019-05-20	41.590000	41.590000	41.680000	39.459999	41.189999
7	2019-05-21	41.500000	41.500000	42.240002	41.250000	42.000000
8	2019-05-22	41.250000	41.250000	41.279999	40.500000	41.049999
9	2019-05-23	40.470001	40.470001	41.090000	40.020000	40.799999

	Volume
0	186322500
1	79442400
2	46661100
3	36086100
4	38115500
5	20225700
6	29222300
7	10802900
8	9089500
9	11119900

```
[23]: min_val = data['Volume'].min()
max_val = data['Volume'].max()
data['Volume'] = 20 + ((data['Volume'] - min_val) / (max_val - min_val)) * 70
data = data.drop('Date', axis=1)
data.head(10)
```

	Adj Close	Close	High	Low	Open	Volume
0	41.570000	41.570000	45.000000	41.060001	42.000000	55.488184
1	37.099998	37.099998	39.240002	36.080002	38.790001	34.754999
2	39.959999	39.959999	39.959999	36.849998	38.310001	28.395904
3	41.290001	41.290001	41.880001	38.950001	39.369999	26.344508
4	43.000000	43.000000	44.060001	41.250000	41.480000	26.738182
5	41.910000	41.910000	43.290001	41.270000	41.980000	23.267821
6	41.590000	41.590000	41.680000	39.459999	41.189999	25.013030
7	41.500000	41.500000	42.240002	41.250000	42.000000	21.439935
8	41.250000	41.250000	41.279999	40.500000	41.049999	21.107560
9	40.470001	40.470001	41.090000	40.020000	40.799999	21.501428

```
[24]: data.isnull().sum() / len(data) * 100
```

```
[24]: Adj Close    3.047091
Close          0.000000
High           0.000000
Low            0.000000
Open           0.000000
Volume         0.000000
dtype: float64
```

```
[25]: ac_mean = np.mean(data['Adj Close'])
data.loc[data.loc[:, 'Adj Close'].isna(), 'Adj Close'] = ac_mean
```

```
ac_mean
```

```
[25]: np.float64(44.45505001335714)
```

```
[26]: def numeric(temp):  
    print("Count =", len(temp))  
    print("Sum =", sum(temp))  
    print("Range =", max(temp) - min(temp))  
    print("Min =", min(temp))  
    print("Max =", max(temp))  
    print("Mean =", st.mean(temp))  
    print("Median =", st.median(temp))  
    print("Variance =", st.variance(temp))  
    print("Standard Deviation =", np.std(temp))  
for column in data.columns:  
    print(f"Column: {column}")  
    numeric(data[column])  
    print("-" * 100)
```

Column: Adj Close

Count = 1444

Sum = 64193.09221928771

Range = 71.51999665

Min = 14.81999969

Max = 86.33999634

Mean = 44.455050013357145

Median = 41.405000689999994

Variance = 242.84568013717615

Standard Deviation = 15.578109784515282

Column: Close

Count = 1444

Sum = 64208.00002281

Range = 71.51999665

Min = 14.81999969

Max = 86.33999634

Mean = 44.46537397701523

Median = 41.35500145

Variance = 243.18099180820852

Standard Deviation = 15.588860892289233

Column: High

Count = 1444

Sum = 65395.16608429

Range = 69.20000076

Min = 17.79999924
Max = 87.0
Mean = 45.287511138704986
Median = 41.90999985
Variance = 247.77355643331714
Standard Deviation = 15.735373147715215

Column: Low
Count = 1444
Sum = 63020.92695994
Range = 70.47000027
Min = 13.71000004
Max = 84.18000031
Mean = 43.64330121879502
Median = 40.518499375000005
Variance = 239.71252758757373
Standard Deviation = 15.477290514881542

Column: Open
Count = 1444
Sum = 64247.92395101
Range = 69.67999935
Min = 15.96000004
Max = 85.63999939
Mean = 44.493022126738225
Median = 41.21500015
Variance = 244.73368924377536
Standard Deviation = 15.638548720372972

Column: Volume
Count = 1444
Sum = 34739.44626334135
Range = 70.0
Min = 20.0
Max = 90.0
Mean = 24.057788271012015
Median = 23.295744956793897
Variance = 11.843674248848039
Standard Deviation = 3.4402721198142157

```
[ ]: for column in data.columns:  
      print(f"Column: {column}")
```

```
print("Unique Values:", data[column].unique())
print("-" * 100)
```

Column: Adj Close

Unique Values: [41.56999969 37.09999847 39.95999908 ... 67.29000092 69.75
64.48000336]

Column: Close

Unique Values: [41.56999969 37.09999847 39.95999908 ... 67.29000092 69.75
64.48000336]

Column: High

Unique Values: [45. 39.24000168 39.95999908 ... 68.84999847 70.19000244
66.75]

Column: Low

Unique Values: [41.06000137 36.08000183 36.84999847 ... 65.1780014 67.69000244
63.5]

Column: Open

Unique Values: [42. 38.79000092 38.31000137 ... 67.08999634 65.30000305
67.69000244]

Column: Volume

Unique Values: [55.48818379 34.75499914 28.3959038 ... 24.25022128 25.89836049
33.84919551]


```
[28]: plt.figure(figsize=(18, 12))

plt.subplot(2, 3, 1)
plt.scatter(range(len(data)), data['Adj Close'], color='blue', alpha=0.7,
            label='Adj Close')
plt.plot(range(len(data)), data['Adj Close'], 'r-', alpha=0.5)
plt.title('Adj Close (Scatter + Line)')
plt.legend()

plt.subplot(2, 3, 2)
plt.boxplot(data['Close'])
plt.title('Close (Box Plot)')
```

```

plt.subplot(2, 3, 3)
plt.hist(data['High'], bins=10, color='purple', alpha=0.7)
plt.title('High (Histogram)')

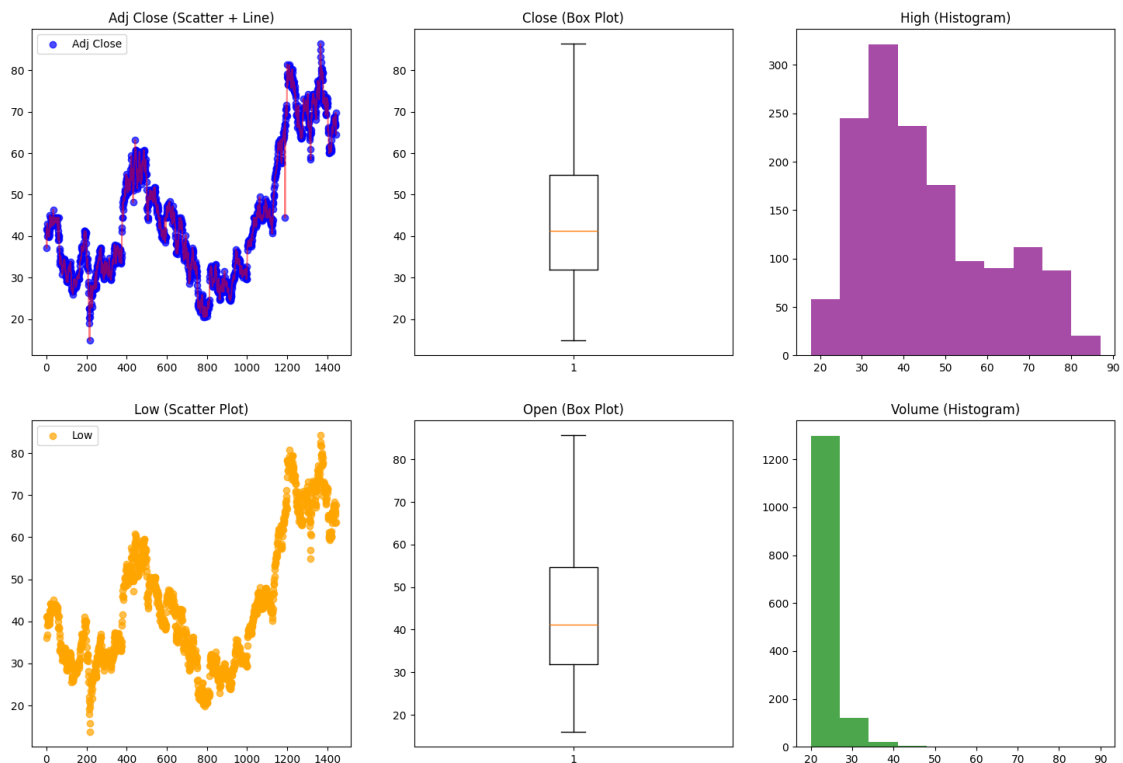
plt.subplot(2, 3, 4)
plt.scatter(range(len(data)), data['Low'], color='orange', alpha=0.7,
            ↪label='Low')
plt.title('Low (Scatter Plot)')
plt.legend()

plt.subplot(2, 3, 5)
plt.boxplot(data['Open'])
plt.title('Open (Box Plot)')

plt.subplot(2, 3, 6)
plt.hist(data['Volume'], bins=10, color='green', alpha=0.7)
plt.title('Volume (Histogram)')

# plt.tight_layout()
plt.show()

```




```
[29]: x = data.iloc[:, 0:-1].values
      y = data.iloc[:, -1].values

      X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
      ↪random_state=1)
```

```
[30]: knn = KNeighborsRegressor(n_neighbors=4, p=2)
      knn.fit(X_train, y_train)
      y_pred = knn.predict(X_test)
      mse = mean_squared_error(y_test, y_pred)
      r2 = r2_score(y_test, y_pred)
      print(f"\nTest MSE with k=4 and Manhattan distance: {mse:.4f}")
      print(f"R2 Score: {r2:.4f}")

      mae = mean_absolute_error(y_test, y_pred)
      rmse = np.sqrt(mse)
      print(f"Mean Absolute Error: {mae:.4f}")
      print(f"Root Mean Squared Error: {rmse:.4f}")

      mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
      print(f"Current Model MAPE: {mape:.2f}%")
```

Test MSE with k=4 and Manhattan distance: 6.0647

R2 Score: 0.1004

Mean Absolute Error: 1.6109

Root Mean Squared Error: 2.4627

Current Model MAPE: 6.48%